
Hands-On Lab - SQL Injection com Entity Framework

Aluno: Carlos

Data: 23/01/2026

Exercício 1 - Login Inseguro

Código:

```
var sql = $"SELECT * FROM Users WHERE Username = '{username}'";
```

Resultado: Ataque com ' OR '1'='1 funcionou. Entrei como admin sem password.

O código junta o texto do utilizador diretamente no SQL. O ' OR '1'='1 torna a condição sempre verdadeira.

Exercício 2 - Login com Parâmetros

Código:

```
new SqlParameter("@username", username)
```

Resultado: Ataque bloqueado.

Os parâmetros separam os dados do código SQL. O ataque é tratado como texto normal.

Exercício 3 - Login com LINQ

Código:

```
.FirstOrDefault(u => u.Username == username)
```

Resultado: Ataque bloqueado.

LINQ gera SQL seguro automaticamente. É a forma mais recomendada.

Exercício 4 - Search com LIKE

Resultado:

- Método inseguro: retornou 5 users
- Método seguro: retornou 0 users

Concatenação de strings permite manipular a query. Parâmetros e LINQ não permitem.

Exercício 5 - Validação

Resultado:

- Método inseguro com "abc": crashou
- Método seguro com "abc": erro tratado

int.TryParse valida antes de processar. int.Parse crasha com input inválido.

Desafio Final - GetByRole

Solução:

```
return _context.Users.Where(u => u.Role == role).ToList();
```

LINQ é seguro por padrão e o código fica mais limpo.

Conclusão

Regras importantes:

- Nunca concatenar strings em SQL
- Usar LINQ sempre que possível
- Se usar SQL manual, usar sempre parâmetros
- Validar todos os inputs

Entity Framework não protege automaticamente. A segurança depende de como o usamos.
